Object-Oriented Programming in Java

01 / Trace execution → 02 / Object contracts → 03 / Data and types → 04 / Change and failure → 05 / Application design

01 / Compilation is one check

Compile-time type checks do not prove intended behavior. Trace numeric conversions, handle runtime failures and test the contract, especially integer division, overflow and empty inputs.

02 / Java passes values

A method receives copies of argument values. A copied reference can reach shared mutable state; reassigning the parameter never reassigns the caller’s variable.

03 / Array boundaries

For a non-null array, valid indices satisfy $0 \le i <$ length. A multidimensional array can have null or unequal-length rows; an outer clone is shallow.

04 / Identity and equality

Use == for reference identity and enum constants; use equals for value equality when the class defines it. Equal objects must have equal hash codes; the reverse implication is false.

05 / Preserve invariants

Construct valid objects and check each operation before invalid state escapes. private and final help, but neither automatically makes a referenced object immutable.

06 / Two method-selection stages

Compile-time argument types select an overload. The runtime receiver chooses an overridden instance implementation. Static methods hide, and changing only a return type does not create an overload.

07 / Choose by collection contract

List preserves order and duplicates, Set models uniqueness, and Map associates keys with values. Compare lookup, insertion and traversal costs rather than assuming one implementation is universally faster.

08 / Generic data flow

Generics are invariant. Use ? extends T for a source of T and ? super T for a destination of T. These are type guarantees, not complete immutability guarantees.

09 / Failure and cleanup

Checked exceptions must be caught or declared. Catch where recovery is meaningful, retain the cause, and use try-with-resources for owned resources. Garbage collection does not replace timely resource cleanup.

10 / Tests and recursion

Test normal cases, boundaries and rejection state. Recursive code also needs a base case and decreasing problem size; memoization avoids repeated states but may leave deep call stacks.

11 / Lambda context

A lambda needs a functional-interface target and captures final or effectively final locals. Captured objects may remain mutable; lambda this refers to the enclosing instance.

12 / Design and diagnose

Separate model rules, event input and rendering. Reproduce defects, inspect the earliest divergence, test a specific hypothesis, and document accepted inputs, effects, outputs and failure behavior.

Systems Programming: C, Processes & Concurrency

01 / Memory and C → 02 / Processes and IPC → 03 / Threads and safety → 04 / Parallel performance

01 / Representation is a contract

unsigned int and wider wrap modulo max+1, but narrower unsigned types promote to int first; signed overflow is undefined. sizeof returns size_t in C bytes. Initialize automatic objects and match formatted-I/O conversions to actual argument types.

02 / Pointers need live bounds

Array arithmetic stays within the array or its one-past position; never dereference the latter. A multidimensional array decays to a pointer to a row, not a pointer to pointer.

03 / Text needs capacity and termination

Reserve strlen+1 bytes. Bounded reads can split a logical line. Check conversion results, suffixes and range; never treat arbitrary bytes as a terminated C string.

04 / Own every allocation

Check n≤SIZE_MAX/sizeof(*p) before multiplying. Check allocation results, initialize before reading, preserve the old pointer on realloc failure and end all borrowing before one eventual free.

05 / Build stages differ

Preprocessing expands tokens; compilation checks a translation unit; linking resolves external definitions. Headers describe interfaces. Macros can duplicate side effects, and include guards do not prevent cross-file multiple definitions.

06 / Signals need a minimal handler

Prefer sigaction and async-signal-safe operations. A volatile sig_atomic_t stop flag can defer cleanup to normal flow. Use an atomic mask-and-wait protocol when checking a predicate and sleeping for signals.

07 / Process lifecycle

fork duplicates execution with separate ordinary memory; exec replaces the image; waitpid collects status. Check each result, close unused inherited descriptors and decode wait status with its matching macros.

08 / Streams are not messages

Pipes and stream sockets can split or combine writes across reads. Define framing and loop over partial transfers. Pipe EOF requires drained data and every write endpoint closed.

09 / Shared storage needs a protocol

Shared memory removes some copying, not synchronization requirements. Use offsets across mappings. Semaphore permits track resources; mutexes protect invariants. Define initialization, shutdown and recovery ownership.

10 / Thread coordination

Keep arguments alive through use and join created threads correctly. Protect every conflicting access. A condition wait belongs in a predicate loop with its mutex; volatile is not general synchronization.

11 / Prevent circular waits

Acquire locks by stable global rank and include joins, condition waits and I/O in dependency analysis. A trylock loop can avoid blocking yet still livelock; timeouts alone do not diagnose deadlock.

12 / Measure equivalent work

Speedup=T_1/Tp using elapsed time for the same verified result. Partition output ownership, combine after dependencies complete and account for serial work, bandwidth, imbalance and synchronization costs.

Algorithm Design and Analysis

01 / Analysis and structures → 02 / Graph algorithms → 03 / Greedy proofs → 04 / Divide and randomize

01 / Name the guarantee

Worst-case maximizes over inputs; expected averages over stated randomness; amortized bounds the worst-case total of any sequence, divided by its length (no probability). Input size and arithmetic cost model must be explicit before interpreting any bound.

02 / Representation tradeoffs

Array indexing is O(1), interior shifts O(n). Doubly linked edits at known positions are O(1), but locating an index is O(n). Include both search and modification costs.

03 / BST versus AVL

BST search and update cost O(h); range reporting costs O(h+k). AVL maintains child-height difference at most one, ensuring h=O(log n). k is output size.

04 / Heap priorities

A min-heap keeps each parent no larger than its children. Minimum is O(1); insert and remove-min are O(log n). A heap array is not fully sorted.

05 / Hash-table assumptions

With suitable uniform hashing, chaining lookup is expected O(1+α), α=n/m. Linear probing needs tombstones or cluster repair after deletion. Cuckoo insertion can trigger expensive rebuilding.

06 / Graph traversals

Adjacency-list BFS and DFS cost O(n+m). BFS minimizes edge count. In undirected DFS, the tree edge from parent u to child v is a bridge iff low[v]>disc[u]; skip only that parent edge (by edge id).

07 / Path versus tree

Dijkstra relaxes d[u]+w(u,v) and requires nonnegative weights. Prim uses w(u,v) to connect a new vertex. MSTs minimize total network weight, not source-to-vertex distances.

08 / Greedy proof certificates

Fractional knapsack uses density and equal-weight exchange. Interval partitioning matches overlap depth. Huffman combines two minimum frequencies. Each proof depends on its specific feasible-choice structure.

09 / Recurrences

T(n)=T(n/2)+Θ(1) gives Θ(log n); 2T(n/2)+Θ(n) gives Θ(n log n); 3T(n/2)+Θ(n) gives Θ(n^log₂3). Include split and combination costs.

10 / Sort versus select

Randomized quicksort sorts both sides in expected O(n log n). Quickselect follows one side in expected O(n). Both can take $\Theta(n^2)$ on unlucky pivot sequences.

11 / Random structures

Fisher-Yates chooses j uniformly in {0,…,i} before fixing position i. Skip-list promotion probability one half gives expected O(log n) operations and expected O(n) storage.

12 / Read the input length

For q encoded in k≈log₂q bits, √q≈2^(k/2). A polynomial in the numerical value may be exponential in bit length; this distinction matters for cryptographic complexity.

Models of Computation: Languages, Machines & Logic

01 / Finite automata → 02 / Grammars and parsing → 03 / Computability → 04 / Logic and inference

01 / Three empty objects

ε is the empty string; {ε} is the language containing it; ∅ is the language with no strings. Every Kleene star includes ε.

02 / DFA design

Name each state by what it says about the processed prefix. Prove initialization and preservation, then connect the final state to acceptance after all input is consumed.

03 / NFA acceptance

One successful complete run suffices. Epsilon moves consume nothing. Swapping NFA final states does not complement its language; first determinize and complete the machine.

04 / Subset construction

Start with epsilon closure. For each symbol, union all possible moves and close again. Accept subsets intersecting F. At most 2^m subsets arise from m NFA states.

05 / Regular equivalence

Build a product DFA accepting disagreement, then test accepting-state reachability. Empty symmetric difference means equal languages; a reachable disagreement path supplies a counterexample.

06 / CNF and CYK

Grammar CNF uses A→BC or A→a, with a protected start epsilon exception. CYK processes increasing spans; T[i,j] means w[i:j], and splits satisfy i<k<j.

07 / Recognize versus decide

A recognizer accepts every member but may loop outside its language. A decider must halt on every input. A timeout alone establishes neither rejection nor undecidability.

08 / Complement theorem

L is decidable exactly when L and its complement are recognizable. Alternate the two recognizers fairly; whichever accepts determines the correct terminating answer.

09 / Three logical classifications

Satisfiable means true somewhere; valid means true everywhere; unsatisfiable means true nowhere. With k atoms, exhaustive truth-table testing uses 2^k assignments.

10 / Entailment test

Γ entails F exactly when the premises together with ¬F are unsatisfiable. A countermodel makes every premise true and the conclusion false.

11 / Quantifier discipline

All A are B: ∀x(A(x)→B(x)). Some A are B: ∃x(A(x)∧B(x)). ¬∀xP≡∃x¬P and ¬∃xP≡∀x¬P, so "not all A are B" is ∃x(A(x)∧¬B(x)); mixed quantifier order controls witness dependence.

12 / Impossibility proofs

Assume a machine exists, then force a contradiction using repeated states or self-reference. One nonterminating implementation does not prove undecidability; the claim must exclude every possible decider.

Artificial Intelligence: Search, Reasoning & Learning

01 / Search and games → 02 / Learning from data → 03 / Neural models → 04 / Probability and clusters

01 / Match the frontier to the objective

BFS minimizes depth; UCS minimizes accumulated nonnegative path cost. Equal positive step costs align them. Test a UCS goal when removed from the priority queue, and account for duplicate states.

02 / A* needs a justified heuristic

Use f=g+h. Admissibility is a lower bound; consistency also constrains every edge. Graph search with inconsistent estimates may need reopening. Beam truncation sacrifices ordinary A* guarantees.

03 / Optimization is budgeted exploration

Hill climbing can stop locally. Annealing accepts a worsening cost Δ with exp(-Δ/T). Genetic search needs valid encodings and diversity. A finite stochastic run is evidence of a found solution, not global optimality.

04 / Game backups

MAX takes maxima, MIN minima, chance nodes probability-weighted means. Alpha-beta prunes when alpha reaches beta for the same minimax tree. Heuristic cutoffs replace exact terminal guarantees with approximations.

05 / Prevent leakage

Fit preprocessing on training data only (refit per fold); make model choices on validation data. Keep the final test untouched. Use groups or time order when independent random splitting would misrepresent deployment.

06 / Evaluate the relevant errors

Precision=TP/(TP+FP), recall=TP/(TP+FN), F1=2TP/(2TP+FP+FN). Name the positive class, handle empty denominators and weight errors when their consequences differ.

07 / Tree splits and pruning

Information gain is parent entropy minus size-weighted child entropy. Unique identifiers can look excellent in training. Validate numeric thresholds, depth and pruning; gain ratio is a mitigation, not a guarantee.

08 / Backpropagation bookkeeping

Save a forward pass, compute all gradients with those parameter values, then update. For $L=\frac{1}{2}\Sigma(a-t)^2$ and sigmoid units, output $\delta=(a-t)a(1-a)$; hidden $\delta_j=a_j(1-a_j)\Sigma_k w_{kj}\delta_k$.

09 / Regularize deliberately

Learning rate affects stability; zero gradient need not indicate a minimum. Choose early stopping on validation. Inverted dropout scales retained training activations by 1/q (q = keep probability) and uses no dropout at inference.

10 / Margins and committees

Soft SVM balances norm against C-weighted slack; kernels must be valid inner products. Bagging reduces variance through resampling; boosting adapts emphasis; correlated errors limit the benefit of voting.

11 / Probability bookkeeping

Joint probabilities multiply local Bayesian-network factors. Sum hidden variables before normalizing. Conditioning differs from causation, and absence of a direct edge does not imply unconditional independence.

12 / Clustering choices change the answer

K-means pairs arithmetic means with squared Euclidean error. Single linkage can chain; complete linkage tests the farthest cross pair. Internal quality scores and cluster counts need interpretation and stability checks.